CS 0007: Introduction to Java Lecture 18

Nathan Ong
University of Pittsburgh
November 8, 2016

Main Portions of OOP

- Building a class
- Determining inter-class relationships

Class Components

- Class-level/Instance Variables
- Methods
- Constructor(s)

Inter-class Relationships

- Building a single class is useless. It needs to be used in context.
- There are two main relationships between classes
  - Sub/Super class
  - Ownership

Modeling a Problem

"Joe's Automotive Shop services foreign cars, and specializes in servicing cars made by Mercedes, Porsche, and BMW. When a customer brings a car to the shop, the manager gets the customer's name, address, and telephone number. Then the manager determines the make, model, and year of the car, and gives the customer a service quote. The service quote shows the estimated parts charges, estimated labor charges, sales tax, and total estimated charges."

Source: Starting Out with Java: From Control Structures through Objects by Tony Gaddis

Find the Nouns

"Joe's Automotive Shop services foreign cars, and specializes in servicing cars made by Mercedes, Porsche, and BMW. When a customer brings a car to the shop, the manager gets the customer's name, address, and telephone number. Then the manager determines the make, model, and year of the car, and gives the customer a service quote. The service quote shows the estimated parts charges, estimated labor charges, sales tax, and total estimated charges."

Coalescing

address
BMW
car
quote
cars
customer
estimated labor charges
charges
estimated parts charges

foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name

Porsche
sales tax
service
shop
telephone number
total estimated
year

Noun Out of Scope

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Subtypes

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Whether these subtypes are specific objects (instances) or more specific types of cars (subclasses) will depend on your needs. In this scenario, there is no need to represent them as classes.

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe’s Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Simple properties can be composed of primitives or preexisting object definitions, and fall under a particular object class.

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A car had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A car had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A customer had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A customer had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A service quote had four simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A service quote had four simple properties in our scenario. What types would they be?

Who Owns Everything?

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Our scenario revolves around the automotive shop, so this will hold the main method and will be the overarching class.

Code on the board!

The `Object` Class

- Let us examine the API for the highest class.

- **`toString()`** looks interesting. “It is recommended that all subclasses override this method.”

What Is `toString()`?

- It provides a **`String`** representation of an object
- It should be descriptive, not just what type it is, but what is contained within it.

Who is Responsible?

- How do we determine which class(es) should have which functions?
- You need to ask: Who’s state is needed or changing?

Easy examples

- Using the automotive shop example, we have four classes, Shop, Customer, Car, and Service Quote.
- Who issues the service quotes?
- Who deals with customers?

Medium example

- Using the automotive shop example, we have four classes, Shop, Customer, Car, and Service Quote.
- Who does the payment of the service quote?
  - Customer?
  - Shop?
  - Both?

Hard example

- Using the automotive shop example, we have four classes, Shop, Customer, Car, and Service Quote.
- Who deals with car alterations?
  - Shop?
  - Mechanic?
  - Car?